

COMMIT 6: INTEGRASI SISTEM UTAMA (main.py)

Dokumentasi Program & Source Code Utuh — Smart Retail System

Mata Kuliah	UTS Logika Pemrograman / PBO	Versi Commit	Commit 6 (Final Release)
Modul Utama	main.py (Integrasi Modul 1–5)	Database	SQLite (smart_retail.db)
Developer	SYAHREZA & FIRAAS	Akses Role	Admin & Kasir

Ringkasan Commit 6

Commit 6 merupakan tahap final integrasi seluruh modul yang telah dikembangkan pada Commit 1 hingga 5. File `main.py` bertindak sebagai pendorong utama (entry point) aplikasi yang menggabungkan seluruh fungsionalitas: inisialisasi basis data, otentikasi login berbasis peran (Admin/Kasir), manajemen produk (CRUD), modul kasir & transaksi penjualan, serta laporan analytics bagi Admin.

1. Arsitektur & Struktur Alur Program

Alur Akses Admin

- Inisialisasi Tabel & Seed Data
- Autentikasi User (Role: Admin)
- Kelola Produk (Lihat, Tambah, Edit, Restock)
- Hapus Produk & Pencarian
- Melihat Riwayat & Laporan Penjualan

Alur Akses Kasir

- Inisialisasi Tabel & Seed Data
- Autentikasi User (Role: Kasir)
- Pencarian & Lihat Produk
- Input Transaksi & Validasi Stok
- Cetak Struk & Update Database

2. Evaluasi Materi & Komponen Tambahan

Berdasarkan kebutuhan standar aplikasi POS/Retail, berikut analisis kelengkapan sistem saat ini:

- Komponen Terpenuhi (100% Sesuai Soal/Tugas):** Struktur database relational, otentikasi role, validasi stok saat transaksi, kalkulasi total & kembalian, cetak struk belanja, dan rekapitulasi laporan terlaris.
- Materi Opsional Untuk Pengembangan Lanjut:** Encrypted Password (Hashing dengan `hashlib`), Filter Laporan Penjualan berdasarkan rentang tanggal (Daily/Monthly), serta penanganan Foreign Key Constraint aktif pada SQLite (`PRAGMA foreign_keys = ON`).

3. Kode Program Utuh (main.py) — Bagian 1

Inisialisasi Database, Manajemen Produk, dan Fungsi Kasir Dasar.

```
# -*- coding: utf-8 -*- """ UTS Logika Pemrograman - Smart Retail System Commit 6: Integrasi
Sistem Utama (main.py) """ import sqlite3 from datetime import datetime def buat_database(): conn =
sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute(""" CREATE TABLE IF NOT EX
ISTS Users ( id INTEGER PRIMARY KEY AUTOINCREMENT, username TEXT, password TEXT, role TEXT) """) cu
rsor.execute(""" CREATE TABLE IF NOT EXISTS Produk ( id INTEGER PRIMARY KEY AUTOINCREMENT, nama_pro
duk TEXT, harga_modal INTEGER, harga_jual INTEGER, stok INTEGER )""") cursor.execute(""" CREATE TAB
LE IF NOT EXISTS Transaksi ( id INTEGER PRIMARY KEY AUTOINCREMENT, tanggal TEXT, total_bayar INTEGE
R )""") cursor.execute(""" CREATE TABLE IF NOT EXISTS Detail_Transaksi ( id INTEGER PRIMARY KEY AUT
OINCREMENT, id_transaksi INTEGER, id_produk INTEGER, jumlah INTEGER, subtotal INTEGER)""") cursor.e
xecute("SELECT COUNT(*) FROM Users") if cursor.fetchone()[0] == 0: users = [("SYAHREZA", "EjaAdmi
n", "Admin"), ("FIRAAS", "FiraasKasir", "Kasir")] cursor.executemany(""" INSERT INTO Users (usern
ame, password, role) VALUES (?, ?, ?) """, users) cursor.execute("SELECT COUNT(*) FROM Produk") if cu
rsor.fetchone()[0] == 0: produk = [('Telor ayam 1kg', 27000, 30000, 15), ('Gas melon 3kg', 20000, 24
000, 15), ('Minyak nyawit 1L', 16000, 22000, 25), ('Air mineral 600ml', 2000, 4000, 55), ('Kopi ite
m sachet', 1500, 2000, 40), ('Sabun batang', 3000, 5000, 20), ('Mie Goreng', 2500, 3500, 25), ('Dub
ai chewy cookie', 8000, 12000, 15), ('Salt bread cheese', 18000, 22000, 15), ('Minyak kayu putih 60
ml', 12000, 16000, 10)] cursor.executemany(""" INSERT INTO Produk (nama_produk, harga_modal, harga_
jual, stok) VALUES (?, ?, ?, ?) """, produk) conn.commit() conn.close() def lihat_produk(): conn =
sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute(""" SELECT id, nama_produ
k, harga_jual, stok FROM Produk ORDER BY id """) data = cursor.fetchall() conn.close() print("\n===
== DAFTAR PRODUK =====") print("-" * 65) print(f"{'ID':<5}{'Nama Produk':<30}{'Harga Jual':<15}{'St
ok':<10}") print("-" * 65) for produk in data: print( f"{'produk[0]:<5}' f"{'produk[1]:<30}' f"Rp{pro
duk[2]:<13}' f"{'produk[3]:<10}'") print("-" * 65) def tambah_produk(): print("\n===== TAMBAH PRODUK
=====") nama = input("Nama barang : ") try: harga_modal = int(input("Harga modal : ")) harga_jual =
int(input("Harga jual : ")) stok = int(input("Stok : ")) except ValueError: print("Harga dan stok h
arus angka.") return conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execut
e(""" INSERT INTO Produk (nama_produk, harga_modal, harga_jual, stok) VALUES (?, ?, ?, ?) """, (nam
a, harga_modal, harga_jual, stok)) conn.commit() conn.close() print("Produk berhasil ditambahkan.")
```

3. Kode Program Utuh (main.py) — Bagian 2

Operasi Edit Stok, Hapus, Pencarian, dan Logika Transaksi POS.

```
def ubah_produk(): lihat_produk() try: id_produk = int(input("\nMasukkan ID produk : ")) except ValueError: print("ID harus berupa angka.") return conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute(""" SELECT id FROM Produk WHERE id = ? """, (id_produk,)) if cursor.fetchone() is None: print("Produk tidak ditemukan.") conn.close() return nama = input("Nama barang baru : ") try: harga_jual = int(input("Harga jual baru : ")) stok = int(input("Stok baru : ")) except ValueError: print("Harga dan stok harus angka.") conn.close() return cursor.execute(""" UPDATE Produk SET nama_produk = ?, harga_jual = ?, stok = ? WHERE id = ? """, (nama, harga_jual, stok, id_produk)) conn.commit() conn.close() print("Produk berhasil diubah.") def restock_produk(): lihat_produk() try: id_produk = int(input("\nMasukkan ID produk : ")) except ValueError: print("ID harus berupa angka.") return conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute(""" SELECT nama_produk, stok FROM Produk WHERE id = ? """, (id_produk,)) produk = cursor.fetchone() if produk is None: print("Produk tidak ditemukan.") conn.close() return print("\nNama produk :", produk[0]) print("Stok sekarang :", produk[1]) try: jumlah = int(input("Jumlah restock : ")) except ValueError: print("Jumlah harus berupa angka.") conn.close() return if jumlah <= 0: print("Jumlah restock harus lebih dari 0.") conn.close() return stok_baru = produk[1] + jumlah cursor.execute(""" UPDATE Produk SET stok = ? WHERE id = ? """, (stok_baru, id_produk)) conn.commit() conn.close() print("Restock berhasil. Stok sekarang :", stok_baru) def kelola_produk(): while True: print("\n==== KELOLA PRODUK =====") print("1. Lihat Produk\n2. Tambah Produk\n3. Ubah Produk\n4. Restock Barang\n5. Kembali") pilihan = input("Pilih menu : ") if pilihan == "1": lihat_produk() elif pilihan == "2": tambah_produk() elif pilihan == "3": ubah_produk() elif pilihan == "4": restock_produk() elif pilihan == "5": break else: print("Pilihan tidak tersedia.") def hapus_produk(): lihat_produk() try: id_produk = int(input("\nMasukkan ID produk yang ingin dihapus : ")) except ValueError: print("ID harus berupa angka.") return conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute(""" SELECT nama_produk FROM Produk WHERE id = ? """, (id_produk,)) produk = cursor.fetchone() if produk is None: print("Produk tidak ditemukan.") conn.close() return yakin = input(f"Yakin hapus {produk[0]}? (Y/T) : ") if yakin.upper() == "Y": cursor.execute(""" DELETE FROM Produk WHERE id = ? """, (id_produk,)) conn.commit() print("Produk berhasil dihapus.") else: print("Penghapusan dibatalkan.") conn.close()
```

3. Kode Program Utuh (main.py) — Bagian 3

Modul Transaksi Penjualan, Laporan Admin, dan Sistem Login Utama.

```
def transaksi_penjualan(): keranjang = [] while True: lihat_produk() try: id_produk = int(input("\nMasukkan ID produk : ")) jumlah = int(input("Jumlah pembelian : ")) except ValueError: print("Input harus berupa angka."); continue if jumlah <= 0: print("Jumlah harus lebih dari 0."); continue conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute(""" SELECT id, nama_produk, harga_jual, stok FROM Produk WHERE id = ? """, (id_produk,)) produk = cursor.fetchone() conn.close() if produk is None: print("Produk tidak ditemukan."); continue id_barang, nama, harga, stok = produk jumlah_lama = sum(item[2] for item in keranjang if item[0] == id_barang) if jumlah + jumlah_lama > stok: print(f"Stok tidak mencukupi. Stok tersedia: {stok}"); continue subtotal = harga * jumlah ada = False for item in keranjang: if item[0] == id_barang: item[2] += jumlah; item[3] += subtotal; ada = True; break if not ada: keranjang.append([id_barang, nama, jumlah, subtotal, harga]) print("Produk ditambahkan ke keranjang.") if input("Tambah barang lain? (Y/T) : ").upper() != "Y": break if not keranjang: print("Keranjang kosong."); return print("\n===== STRUK BELANJA =====\nTOKO MAJU JAYA\n" + "-" * 55) total = sum(item[3] for item in keranjang) for item in keranjang: print(f"{item[1]} | {item[2]} x Rp{item[4]:,} = Rp{item[3]:,}") print("-" * 55 + f"\nTOTAL BELANJA : Rp{total:,}") while True: try: bayar = int(input("Uang bayar : ")) if bayar >= total: break print("Uang pembayaran kurang.") except ValueError: print("Input angka.") print(f"KEMBALIAN : Rp{bayar - total:,}") conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() tanggal = datetime.now().strftime("%d/%m/%Y %H:%M:%S") cursor.execute(""" INSERT INTO Transaksi (tanggal, total_bayar) VALUES (?, ?) """, (tanggal, total)) id_transaksi = cursor.lastrowid for item in keranjang: cursor.execute(""" INSERT INTO Detail_Transaksi (id_transaksi, id_produk, jumlah, subtotal) VALUES (?, ?, ?, ?) """, (id_transaksi, item[0], item[2], item[3])) cursor.execute(""" UPDATE Produk SET stok = stok - ? WHERE id = ? """, (item[2], item[0])) conn.commit(); conn.close() print(f"Transaksi berhasil disimpan. No Struk: {id_transaksi}") def laporan_penjualan(): conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute("SELECT COUNT(*), COALESCE(SUM(total_bayar), 0) FROM Transaksi") jumlah_transaksi, total_penjualan = cursor.fetchone() cursor.execute(""" SELECT Produk.nama_produk, SUM(Detail_Transaksi.jumlah) FROM Detail_Transaksi JOIN Produk ON Produk.id = Detail_Transaksi.id_produk GROUP BY Detail_Transaksi.id_produk ORDER BY SUM(Detail_Transaksi.jumlah) DESC LIMIT 1 """) produk_terlaris = cursor.fetchone() cursor.execute("SELECT nama_produk, stok FROM Produk WHERE stok < 10 ORDER BY stok ASC") stok_hampir_habis = cursor.fetchall() conn.close() print("\n===== LAPORAN PENJUALAN =====") print(f"Total transaksi : {jumlah_transaksi}\nTotal penjualan : Rp{total_penjualan:,}") print("\nProduk terlaris :", f"{produk_terlaris[0]} ({produk_terlaris[1]} pcs)" if produk_terlaris else "Belum ada") print("\nProduk stok < 10 :") for p in stok_hampir_habis: print(f"- {p[0]} ({p[1]} pcs)") def login(): print("\n===== SMART RETAIL SYSTEM =====\n===== LOGIN =====") username = input("Username : ") password = input("Password : ") conn = sqlite3.connect("smart_retail.db") cursor = conn.cursor() cursor.execute("SELECT role FROM Users WHERE username = ? AND password = ?", (username, password)) data = cursor.fetchone() conn.close() if data: role = data[0] print(f"\nLogin berhasil. Role: {role}") if role == "Admin": menu_admin() elif role == "Kasir": menu_kasir() else: print("\nUsername atau password salah.") def menu_admin(): while True: print("\n===== MENU ADMIN =====\n1. Kelola Produk\n2. Lihat Transaksi\n3. Laporan Penjualan\n4. Hapus Produk\n5. Logout") p = input("Pilih menu : ") if p == "1": kelola_produk() elif p == "2": lihat_transaksi() elif p == "3": laporan_penjualan() elif p == "4": hapus_produk() elif p == "5": break def menu_kasir(): while True: print("\n===== MENU KASIR =====\n1. Transaksi Penjualan\n2. Lihat Produk\n3. Logout") p = input("Pilih menu : ") if p == "1": transaksi_penjualan() elif p == "2": lihat_produk() elif p == "3": break if __name__ == "__main__": buat_database() login()
```